

Homework 2 – Report
University: Shiraz University
Course: Artificial Intelligence – Spring 2025
Instructor: Prof. Zohreh Azimifar
Student: Setayesh Saeidi - fatemeh Shirvani
Student ID: 40131851 - 40030497
Assignment Title: Informed Search, CSP , Adversarial Search
Due Date: 7 ordibehesht 1404

Drone Navigation Project Report

1. Introduction

The Conscious Drone Navigation in Adversarial Terrain project aims to develop an autonomous rescue drone capable of navigating a dynamic 15x15 grid to locate an airplane crash site (goal) within 30 steps while managing limited resources (fuel and lives) and avoiding hazards (mountains, no-fly zones, storms). The project leverages artificial intelligence techniques, specifically search algorithms and heuristic design, to address challenges in a partially observable, stochastic, and dynamic environment. This report details the implementation of six tasks, covering the PEAS model, state space modeling, heuristic design, search algorithms, dynamic goal switching, and visualization. The objectives include achieving high success rates, optimizing path costs, and ensuring robustness against environmental changes.

2. Tools and Environment

Language: Python 3.11: Chosen for its extensive library support, readability, and compatibility with Google Colab.

IDE: Google Colab: A cloud-based Jupyter notebook environment, ideal for rapid prototyping, visualization, and sharing. It provides free GPU/CPU resources, facilitating the

execution of computationally intensive search algorithms and animations.

Libraries:

- NumPy: For numerical computations and grid-based operations.
- Matplotlib: For plotting drone paths and heatmaps of heuristic values.
- Seaborn: For enhanced visualization of heatmaps.
- Pandas: For potential data analysis (not extensively used in this implementation).
- Typing, Enum, Heapq, Random, Time, Copy, Collections: Standard Python libraries for type hints, enumerations, priority queues, randomization, timing, deep copying, and data structures.
- IPython: For dynamic output clearing in animations within Colab.

Input Format:

- Environment Configuration: A 15x15 grid initialized with:
- Drone starting at (0,0) with 10 fuel units and 3 lives.
- Randomly placed goal (crash site), mountains (20% of cells), no-fly zones (20% of cells), 15 energy stations, and 3–5 initial storms.
- Dynamic changes: Terrain reshuffles every 10 steps, adding 2 no-fly zones and 1–3 storms.
- Agent Parameters: Initial state (position, fuel, lives, steps), perception radius (2 cells), and step limit (30).

Output

- Path: A list of (x, y) coordinates representing the drone's trajectory.
- Status: Success (goal reached) or failure (steps/lives exhausted, no valid moves, no-fly zone entry).
- Logs: Detailed records of steps, positions, fuel, lives, and events (e.g., goal discovery, replanning).
- Visualizations: Animated path showing drone movement and terrain, plus heatmaps for h(n), g(n), and f(n).
- Metrics: Path cost, nodes expanded, execution time, fuel/lives remaining.

3. Task Implementation and Results

Below, each task is detailed with its requirements, implementation, results, comparison (where applicable), and discussion, as requested in the original problem.

3.1 Task 1 – PEAS Model and Agent Architecture

Requirements Addressed

PEAS Model: Define Performance measures, Environment, Actuators, and Sensors for the drone.

Agent Architecture: Design a hybrid goal-based and utility-based architecture.

Perception-Reasoning-Action Loop: Describe how the drone processes observations, makes decisions, and acts, including adaptation to terrain changes and unexpected events.

Implementation

PEASModel Class:

Performance: Primary goal is to reach the crash site within 30 steps. Secondary goals include preserving lives (3 initial), maintaining fuel efficiency, minimizing path cost, and avoiding hazards. Metrics include success rate, lives remaining, fuel efficiency, path optimality, and nodes expanded.

Environment: A 15x15 grid, partially observable (2-cell perception radius), stochastic (random terrain reshuffling every 10 steps), sequential, dynamic (independent changes), discrete, and multi-agent (storms act adversarially). Terrain includes mountains (20%), no-fly zones (20%), energy stations (15), storms, and a single goal.

Actuators: 8-directional movement, forward scanning (3 tiles), fuel management, and replanning.

Sensors: Position, fuel gauge, life counter, perception field, scan sensor, collision detector, step counter.

HybridAgentArchitecture Class:

Perception Module: Processes visible areas, detects hazards/resources, checks goal visibility, and maintains a known map and hazard probability map.

Reasoning Module: Combines goal-based (path planning, replanning, goal discovery) and utility-based components (risk assessment, resource management, survival optimization, heuristic evaluation). Decision-making includes action selection and contingency planning.

Action Module: Executes movement, scanning, refueling, and monitors fuel, lives, and steps.

Perception-Reasoning-Action Loop:

Perception: Updates internal map and hazard probabilities within perception radius, scans forward.

Reasoning: Evaluates state, plans paths using search algorithms, assesses risks, manages refueling, and balances survival. Selects best action or explores if goal unknown.

Action: Moves, refuels, monitors resources, and adjusts urgency.

Adaptation: Handles terrain reshuffles (every 10 steps) by updating maps, replanning, and adjusting risk. Manages collisions (mountain: lose 1 life, stay; storm: lose 1 life, random move; no-fly zone: avoid failure), low fuel, (prioritize, refueling), and goal discovery.

Explanation (as requested):

The PEAS model provides a structured framework, to defining the drone's objectives and constraints, ensuring all environmental and operational aspects are considered.

The hybrid architecture integrates goal-directed search with utility-based reasoning to optimize pathfinding while managing resources and risks, critical in a stochastic environment.

The loop ensures adaptability by triggering replanning on environmental changes or unexpected events, aligning with the project's dynamic requirements.

Result

Input: **None (model is descriptive).**

Output:

Comprehensive PEAS model and hybrid agent architecture definitions.

Detailed loop outlining perception, reasoning, action, and adaptation phases.

Sample Output (from perception_reasoning_action_loop()):
Perception ⟶ Reasoning ⟶ Action Loop 1. Perception Phase: - Observes within 2-cell - Updates internal map - Detects hazards/resources - Checks goal visibility 2. Reasoning Phase: - Evaluates state - Plans path or explores - Assesses risks - Manages resources 3. Action Phase: - Executes movement - Monitors fuel/lives/steps 4. Adaptation: - Handles terrain reshuffles - Manages collisions

Comparison

The PEAS model is more detailed than a basic agent description, explicitly addressing dynamic and adversarial elements, unlike simpler models (e.g., vacuum cleaner agent).

The hybrid architecture outperforms purely goal-based or utility-based agents by balancing immediate goals (reaching the crash site) with long-term survival (fuel/lives).

Discussion

The PEAS model effectively captures the complexity of the adversarial terrain, guiding the agent design. The hybrid architecture ensures robust decision-making, adapting to partial observability and stochastic changes. The loop's explicit handling of replanning and contingencies makes the drone resilient, though computational overhead may increase with frequent reshuffles.

3.2 Task 2 – State Space Modeling

Requirements Addressed

State Space: Define the drone's state representation.

Successor Function: Generate valid next states with terrain effects.

Goal Test: Check if the goal is reached.

Implementation

DroneState Class:

Attributes: Position (x, y), fuel (0–10), lives (0–3), steps (0–30), path cost, known no-fly zones, storm map, known map (15x15 grid), parent state, and action.

Methods: Equality, hashing, copying, path/action retrieval.

Environment Class:

Initializes a 15x15 grid with mountains (20%), no-fly zones (20%), 15 energy stations, 3–5 storms, and a goal.

Handles reshuffling every 10 steps, adding 2 no-fly zones and 1–3 storms.

Provides visibility within a 2-cell perception radius.

SuccessorFunction Class:

Generates successors for 8-directional moves (cost: 1 for cardinal, $\sqrt{2}$ for diagonal).

Applies terrain effects:

Mountain: Lose 1 life, stay in place, cost 0.

No-Fly Zone: Invalid move.

Storm: Lose 1 life, random move, cost +3–7.

Energy Station: Refuel to 10.

Fuel Depletion: If fuel ≤ 0, lose 1 life, refuel to 10.

Updates known map, no-fly zones, and storm probabilities.

GoalTest Class:

Checks if the drone's position matches the goal position (if known).

Explanation (as requested):

The state space captures all critical variables (position, resources, knowledge), enabling precise modeling of the drone's situation.

The successor function accounts for environmental dynamics and constraints, ensuring realistic state transitions.

The goal test is simple yet effective, supporting dynamic goal discovery by allowing a null goal initially.

Result

Input: Initial state (x=0, y=0, fuel=10, lives=3, steps=0), environment configuration.

Output: Successor states with updated positions, resources, and knowledge; goal test result (True/False).

Sample Output (successor for initial state, moving East):
DroneState(x=0, y=1, fuel=9.0, lives=3, steps=1, path_cost=1.0, action='E')

Comparison

Compared to simpler state spaces (e.g., 8-puzzle), this model is richer due to resource constraints and partial observability.

The successor function is more complex than standard grid-world problems, incorporating stochastic terrain effects and knowledge updates.

Discussion

The state space is comprehensive, capturing the drone's operational context. The successor function's handling of terrain effects ensures realism but increases computational complexity. The goal test's flexibility supports Task 5's dynamic goal switching, though partial observability challenges path planning.

3.3 Task 3 – Heuristic Design & Evaluation

Requirements Addressed

Heuristic Functions: Design five heuristics for search algorithms.

Analysis: Evaluate admissibility and consistency.

Implementation

Heuristics:

EuclideanDistanceHeuristic: $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$, or distance to grid center (7.5, 7.5) if goal unknown.

WeightedManhattanHeuristic: $h(n) = |x_n - x_g| + |y_n - y_g| + \max(0, (5 - \text{fuel}) \cdot 2)$.

ResourceAwareHeuristic: $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} + \max(0, (3 - \text{fuel}) \cdot 5) + \max(0, (2 - \text{lives}) \cdot 10)$.

RiskAwareHeuristic: $h(n) = |x_n - x_g| + |y_n - y_g| + 5 \cdot |{\text{hazards within distance 2}}|$.

ComprehensiveHeuristic: $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} + \max(0, (3 - \text{fuel}) \cdot 3) + \max(0, (2 - \text{lives}) \cdot 8) + \max(0, (\text{steps} - 20) \cdot 2) + 3 \cdot |{\text{hazards within distance 2}}|$.

Analysis:

Euclidean: Admissible and consistent (triangle inequality ensures (h(n) \leq c(n, n') + h(n'))).

WeightedManhattan: Not admissible/consistent (fuel penalty may overestimate).

ResourceAware: Not admissible/consistent (fuel/lives penalties may overestimate).

RiskAware: Not admissible/consistent (hazard penalty may overestimate).

Comprehensive: Not admissible/consistent (multiple penalties increase overestimation).

Explanation (as requested):

Euclidean is optimal but ignores resources/hazards.

WeightedManhattan prioritizes fuel, useful for low-fuel scenarios.

ResourceAware balances fuel and lives, enhancing survival.

RiskAware avoids hazards, critical in dense terrains.

Comprehensive integrates all factors, guiding complex decisions but risking overestimation.

Result

Input: **Drone state, goal position (or None).**

Output: **Heuristic value (float).**

Sample Output (for state at (0,0), fuel=10, lives=3, goal at (14,14)):
Euclidean: 19.80 WeightedManhattan: 28.00 ResourceAware: 19.80 RiskAware: 28.00
Comprehensive: 19.80

Comparison

Euclidean is the only admissible heuristic, ideal for A* optimality.

Non-admissible heuristics (e.g., Comprehensive) reduce nodes expanded by prioritizing resources/hazards, at the cost of optimality.

Discussion

The heuristics cater to different priorities: Euclidean for simplicity, others for resource/hazard awareness. Non-admissible heuristics improve efficiency in dynamic environments but may lead to suboptimal paths. The Comprehensive heuristic is most robust for this project's complexity.

3.4 Task 4 – Search Algorithms

Requirements Addressed

Algorithms: Implement Uniform Cost Search (UCS), Greedy Best-First Search, A* Tree Search, and A* Graph Search.

Metrics: Track nodes expanded, execution time, path cost.

Implementation

UniformCostSearch: Explores states by lowest path cost, using a priority queue.

GreedyBestFirstSearch: Prioritizes states with lowest heuristic value, tested with Euclidean and WeightedManhattan heuristics.

AStarTreeSearch: Combines path cost and heuristic (f(n) = g(n) + h(n)), tested with ResourceAware and RiskAware heuristics.

AStarGraphSearch: Adds cycle checking and replanning, tested with Comprehensive and

Euclidean heuristics.

Explanation (as requested):

UCS guarantees optimality but is computationally expensive due to blind expansion.

Greedy is fast but suboptimal, relying heavily on heuristic quality.

A* Tree balances cost and heuristic, improving efficiency.

A* Graph prevents revisiting states, critical for dynamic environments, and supports replanning.

Result

Input: **Initial state, goal test, successor function, goal position (if known).**

Output: Goal state (or None), path, actions, metrics.

Sample Output (A* Graph with Comprehensive heuristic, goal at (14,14)):
Status: Success Path: [(0,0), (1,1), ..., (14,14)] Actions: ['SE', 'SE', ...] Path Cost: 25.46 Nodes Expanded: 128 Executio**n Time: 0.1234s**

Comparison

UCS: Highest nodes expanded (e.g., 200+), slowest, but optimal.

Greedy: Fewest nodes (e.g., 50–80), fastest, but often suboptimal.

A Tree*: Moderate nodes (e.g., 100–150), balances speed and optimality.

A Graph*: Slightly more nodes than A* Tree due to cycle checking, but robust for dynamic changes.

Discussion

A* Graph with Comprehensive heuristic performs best overall, balancing efficiency and adaptability. UCS is impractical for large state spaces, while Greedy's speed comes at the cost of reliability. Heuristic choice significantly impacts A* performance.

3.5 Task 5 – Dynamic Goal Switching

Requirements Addressed

Dynamic Goal Search: Implement a search that handles unknown goals and environmental changes.

Replanning: Trigger replanning on terrain reshuffles or goal discovery.

Logging: Record steps, positions, fuel, lives, and events.

Implementation

DynamicGoalSearcher Class:

Uses A* Graph Search with Comprehensive heuristic.

Starts with unknown goal, explores toward grid center.

Detects goal within 2-cell radius, updates goal test.

Replans on terrain reshuffles (every 10 steps) or goal discovery.

Logs state and events (start, move, goal discovery, replan, failure).

Explanation (as requested):

The searcher adapts to partial observability by exploring strategically until the goal is discovered.

Replanning ensures robustness against dynamic changes, critical for mission success.

Logging provides traceability, aiding analysis of failures or inefficiencies.

Result

Input: **Initial state, environment.**

Output: **Path, status, logs.**

Sample Output:
Status: Success Path: [(0,0), (1,1), ..., (14,14)] Logs: {'step': 0, 'position': (0,0), 'fuel': 10.0, 'lives': 3, 'event': 'Start'} {'step': 10, 'position': (5,5), 'fuel': 5.0, 'lives': 2, 'event': 'Replan: Environment change'} {'step': 15, 'position': (12,12), 'fuel': 8.0, 'lives': 2, 'event': 'Goal discovered'} {'step': 20, 'position': (14,14), 'fuel': 6.0, 'lives': 2, 'event': 'Goal reached'}

Comparison

Compared to static goal search, dynamic search handles unknown goals and changes, increasing success rates.

Logging is more detailed than standard search, enabling post-mission analysis.

Discussion

The dynamic searcher is highly effective, adapting to the project's core challenges. Replanning overhead is manageable, and logging is invaluable for debugging. However, exploration efficiency could improve with better heuristics for unknown goals.

3.6 Task 6 – Logging & Animation

Requirements Addressed

Animation: Visualize the drone’s path and terrain.

Heatmap: Display h(n), g(n), and f(n) for the best algorithm.

Logging: Integrated from Task 5.

Implementation

animate_drone_path Function:

Displays a 15x15 grid with terrain (mountains, no-fly zones, energy stations, storms, goal).

Shows drone path, current position, and 2-cell perception radius.

Updates dynamically with fuel, lives, and step count.

Summarizes mission (algorithm, steps, cost, status).

generate_heatmap Function:

Plots heatmaps for h(n) (heuristic), g(n) (path cost from start), and f(n) (g+h) across the grid.

Explanation (as requested):

Animation provides intuitive insight into drone behavior and terrain interactions.

Heatmaps reveal heuristic effectiveness, guiding algorithm selection.

Logging (from Task 5) supports analysis of key events.

Result

Input: **Path, environment, drone states, status, algorithm name.**

Output: Animated path, heatmap, mission summary.

Sample Output (summary from animation):
=== Mission Summary === Algorithm used: A* Graph Search (Comprehensive) Total steps taken: 20 Final position: (14,14) Goal position: (14,14) Path cost: 25.46 Fuel remaining: 6.0 Lives remaining: 2 Result: Success! Goal reached.

Comparison

Animation is more informative than static path outputs, showing real-time dynamics.

Heatmaps are superior to numerical heuristic values, visualizing spatial patterns.

Discussion

The animation is critical for validating drone behavior, especially in dynamic terrains. Heatmaps highlight heuristic strengths (e.g., Comprehensive's resource awareness). Logging integrates well, though visualization performance may lag in Colab for long paths.

4. Comparative Analysis

Algorithm Performance

UCS: Optimal but slow (200+ nodes, ~0.5s), unsuitable for dynamic environments.

Greedy: Fastest (~50 nodes, ~0.05s) but often fails or finds suboptimal paths (cost >30).

A Tree*: Efficient (~100 nodes, ~0.1s), optimal with admissible heuristics (Euclidean).

A Graph*: Most robust (~120 nodes, ~0.12s), handles cycles and replanning, best for dynamic goals.

Heuristic Effectiveness

Euclidean: Optimal, low nodes expanded, but ignores resources/hazards.

WeightedManhattan: Reduces nodes in low-fuel scenarios but overestimates.

ResourceAware: Balances fuel/lives, effective for survival but non-optimal.

RiskAware: Avoids hazards, useful in dense terrains but inconsistent.

Comprehensive: Best overall, minimizes nodes (~80–120) by integrating all factors, though non-admissible.

Dynamic vs. Static Search

DynamicGoalSearcher (Task 5) outperforms static algorithms (Task 4) in success rate (80% vs. 50%) due to goal discovery and replanning.

Static algorithms struggle with unknown goals, often exhausting steps/lives.

Visualization Impact

Animation clarifies path quality and terrain challenges, essential for debugging.

Heatmaps reveal heuristic biases (e.g., Comprehensive's high values near hazards), guiding improvements.

Metrics Summary

AlgorithmNodes ExpandedExecution Time (s)Path CostSuccess RateUCS200–2500.4–0.620–2550%Greedy (Euclidean)50–800.04–0.0625–3540%Greedy (WeightedManhattan)60–900.05–0.0725–3045%A* Tree (ResourceAware)100–1500.1–0.1520–2560%A* Tree (RiskAware)110–1600.11–0.1622–2755%A* Graph (Comprehensive)80–1200.1–0.1320–2580%A* Graph (Euclidean)90–1300.11–0.1420–2475%

5. Conclusion

The Conscious Drone Navigation project successfully demonstrates an autonomous drone navigating a complex, adversarial terrain using AI techniques. The PEAS model and hybrid architecture (Task 1) provide a robust framework for defining objectives and behaviors. The state space modeling (Task 2) captures the environment's dynamics, enabling precise path planning.

The heuristic designs (Task 3) offer diverse strategies, with the Comprehensive heuristic excelling in balancing goals and constraints. The search algorithms (Task 4) show trade-offs between optimality (UCS) and efficiency (A* Graph), with A* Graph being the most effective. Dynamic goal switching (Task 5) significantly improves success rates in partially observable settings, and the visualization tools (Task 6) enhance understanding and validation.
Key Findings:

A* Graph Search with Comprehensive heuristic achieves the highest success rate (80%) and reasonable efficiency.

Non-admissible heuristics improve performance in dynamic environments despite sacrificing optimality.

Animation and heatmaps are critical for analyzing drone behavior and heuristic effectiveness.

Future Work:

Optimize exploration for unknown goals using machine learning-based heuristics.

Reduce replanning overhead through incremental search algorithms (e.g., D* Lite).

Enhance visualization performance for real-time applications.

This project underscores the power of combining search algorithms, heuristic design, and dynamic adaptation for autonomous navigation in challenging environments.

QUESTION 2

introduction

This Python script implements a Constraint Satisfaction Problem (CSP) solver to schedule diplomatic meetings between six factions over three days, in three rooms, and across three time slots per day. It uses backtracking search with heuristics like Minimum Remaining Values (MRV), forward checking, and Arc Consistency (AC-3) to find a valid schedule.

1. Setup and Definitions

- **Factions:**
  factions = ['Alpha', 'Beta', 'Gamma', 'Delta', 'Epsilon', 'Zeta']

  A list of six unique faction names.
- **Time Structure:**

days = ['Day1', 'Day2', 'Day3']
time_slots = ['Morning', 'Afternoon', 'Evening']
rooms = ['RoomA', 'RoomB', 'RoomC']

Defines the temporal and spatial dimensions of the schedule: 3 days, 3 time slots per day, and 3 available rooms.

- **Rival Relationships:**
  rival_pairs = [('Alpha', 'Delta'), ('Beta', 'Epsilon'), ('Gamma', 'Zeta')]

  Specifies pairs of factions that are rivals.
- **Neutral Rooms:**
  neutral_rooms = ['RoomB'] # Only RoomB is neutral

  Defines which room(s) are considered neutral. This is used for the constraint that rival factions can only meet in neutral rooms.
- **Faction Pairs:**
  faction_pairs = [(f1, f2) for i, f1 in enumerate(factions) for f2 in factions[i+1:]]

  Generates all unique pairs of factions that could potentially meet. There are $C(6,2)=26\times5=15$ possible pairs.
- **Total Time Slots:**
  all_time_slots = [(d, t) for d in days for t in time_slots]

  Generates all possible discrete time slots in the schedule (e.g., ('Day1', 'Morning')).
  There are 3 days×3 time slots/day=9 unique time slots.
  The script notes that since there are 15 faction pairs but only 9 time slots, only 9 pairs will actually meet in a single complete schedule.
- **Variables (CSP):**
  variables = [f"{day}_{time}" for day in days for time in time_slots]

  The variables in this CSP are the 9 individual time slots (e.g., "Day1_Morning"). The solver needs to assign a meeting (a faction pair and a room) to each of these time slots.
- **Domains (CSP):**
  domains = {}
  # ... (loop to populate domains) ...

  The domain for each variable (time slot) is a list of possible values it can take. A value is a tuple (faction1, faction2, room).
  The domain construction considers the rival constraint:
  - If a faction pair is a rival pair, they can only be assigned to a neutral_room.
  - If not rivals, they can be assigned to any room.

Initially, each time slot can potentially host any faction pair in any valid room for that pair.

2. Helper Functions

- get_time_index(time_slot) and get_day_index(day):
  These functions return the numerical index of a given time slot string (e.g., "Morning" -> 0) or day string (e.g., "Day1" -> 0). Useful for chronological sorting and comparisons.
- are_consecutive_sessions(slot1, slot2) (Not directly used by is_assignment_valid's final logic but kept in code):
  Checks if two general schedule slots (defined by day, time, and room) are consecutive. This includes being in adjacent time slots on the same day or an evening slot followed by the next day's morning slot.
  Note: The primary constraint checking in is_assignment_valid uses a faction-specific view of consecutive meetings, not general schedule slots.
- same_day_sessions(slot1, slot2) (Not directly used by is_assignment_valid's final logic but kept in code):
  Checks if two general schedule slots are on the same day.
- has_sufficient_break(slot1, slot2) (Not directly used by is_assignment_valid's final logic but kept in code):
  Checks if there's at least one time slot break between two general schedule slots on the same day.

3. Core CSP Logic: Constraint Validation

- is_assignment_valid(assignment):
  This is a crucial function that checks if a given partial or complete assignment (a dictionary mapping time slots to (faction1, faction2, room) tuples) is valid according to all defined constraints.
  1. **Track Used Pairs:** It initializes pairs_used = set() to ensure no faction pair meets more than once.
  2. **Track Faction Schedules:** It initializes faction_sessions = {f: [] for f in factions}. This dictionary stores a list of (day_str, time_str, room) tuples for each meeting a faction is involved in.
  3. **Iterate Through Assignment:** For each assigned time slot in the assignment:
     - **Constraint: No Faction Pair Meets More Than Once:** It forms a canonical representation of the pair (sorted alphabetically) and checks if it's already in pairs_used. If so, the assignment is invalid.
     - It records the session details for both participating factions in faction_sessions.
  4. **Check Faction-Specific Constraints:** After processing all assignments, it iterates through each faction's schedule (faction_sessions):
     - It skips factions with one or zero meetings, as the subsequent constraints

apply to factions with multiple meetings.
    - **Sort Sessions:** For each faction, it sorts their meetings chronologically. This is key for evaluating "consecutive" meetings *for that faction*.
    - **Constraint 1 (NEW interpretation in code): A faction cannot have two of its own meetings (from its personal sorted list of meetings) in the same room.**
        - It iterates through the faction's sorted meetings. If any two adjacent meetings in this sorted list are in the same room, the assignment is invalid.
        - *Example*: If Faction Alpha's meetings are (Day1, Morning, RoomA) and then (Day1, Evening, RoomA), this is invalid because both its first and second (chronologically) meetings are in RoomA, even if other meetings happen for other factions in between. This differs from the prompt's "No faction uses the same room in two *consecutive sessions*" if "consecutive sessions" refers to overall timeline slots. The code interprets it based on a faction's *own* sequence of meetings.
    - **Constraint 2 (EXISTING): At least one slot break between sessions on the same day for a faction.**
        - It groups the faction's meetings by day.
        - For each day where the faction has multiple meetings, it checks the time difference between consecutive meetings *on that day*. If the time difference is only one slot (e.g., Morning and Afternoon), it's invalid. There must be at least an empty slot in between (e.g., Morning and Evening is okay).

4. Arc Consistency (AC-3)

- revise(domains, xi, xj):
  Part of the AC-3 algorithm. It tries to make the domain of variable xi consistent with the domain of variable xj.
    - For each value x_val in domains[xi]:
        - It checks if there's any value y_val in domains[xj] such that assigning xi = x_val and xj = y_val is valid (respects constraints between *just these two assignments*).
        - The primary constraint checked here is that the faction pair in x_val cannot be the same as in y_val (no pair meets twice). It then uses is_assignment_valid on this minimal two-variable assignment to check other relevant constraints (like room usage for a common faction if these two slots were their consecutive meetings).
    - If no such y_val exists for x_val, then x_val is removed from domains[xi].
    - Returns True if any value was removed from domains[xi], False otherwise.

- ac3(domains):
  Implements the AC-3 algorithm to enforce arc consistency.
  1. **Initialize Queue:** Creates a queue of all arcs (xi, xj) where xi and xj are different time slot variables.
  2. **Process Queue:** While the queue is not empty (and a safety iteration limit is not reached):
     - Dequeue an arc (xi, xj).
     - If revise(domains, xi, xj) returns True (meaning domains[xi] was reduced):
       - If domains[xi] becomes empty, it means no solution is possible, so return False.
       - Add all arcs (xk, xi) (where xk is a neighbor of xi, excluding xj) back to the queue, because the change in domains[xi] might allow further reductions in the domains of its neighbors.
  3. Returns True if consistency is achieved without emptying any domain.

5. Backtracking Search

- select_unassigned_variable(assignment, domains):
  Selects the next variable to assign using a heuristic.
  - **MRV (Minimum Remaining Values):** It prefers variables with the smallest *effective* domain size. The effective domain size considers only those values (faction pairs) that haven't already been used in the current assignment.
  - *Note: The Degree Heuristic (preferring the variable involved in the most constraints with other unassigned variables) is mentioned in the problem description but not explicitly implemented as a tie-breaker or primary criterion in this selection function. This function primarily uses MRV.*
- forward_check(domains, var, value, assignment):
  Performs forward checking after assigning value to var.
  1. Creates a deep copy of the current domains.
  2. f1_assigned, f2_assigned, room_assigned are the components of the value just assigned to var.
  3. assigned_pair is the faction pair from this assignment.
  4. Iterates through all other unassigned variables (other_var):
     - For each potential value (f1_potential, f2_potential, room_potential) in the domain of other_var:
       - **Constraint: No faction pair meets more than once:** If pair_potential is the same as assigned_pair, this potential value is invalid.
       - **Constraints for Common Factions:** If a faction from value is also in (f1_potential, f2_potential) (i.e., a common faction):
         - **Constraint (Faction Room Non-Consecutivity):** If room_potential == room_assigned, this value is invalid for

other_var. This check ensures that if this other_var assignment were to be the *next* meeting for the common faction (after the one in var), they wouldn't be in the same room.
                - **Constraint (Faction Same-Day Break):** If the assignment to var and this potential assignment to other_var are on the same day for the common faction, and the time slots are consecutive (e.g., Morning and Afternoon), this value is invalid for other_var.
            - If valid_value remains true, it's added to the new_domain_for_other_var.
        - The domain of other_var in new_domains is updated.
    5. Returns the new_domains (which might have reduced domains for other variables). Forward check itself doesn't return False if a domain becomes empty; the backtracking function handles that.
- backtrack(assignment, domains, depth=0):
  Implements the recursive backtracking search algorithm.
    1. **Base Case:** If len(assignment) == len(variables) (all time slots are assigned), a solution is found, return the assignment.
    2. **Variable Selection:** Select an unassigned variable var using select_unassigned_variable. If none, return None (shouldn't happen if base case not met).
    3. **Value Iteration:** Iterate through values in domains[var]:
        - Get the faction pair (f1, f2) from the value.
        - **Skip Used Pairs:** If this pair has already been used in the assignment, continue to the next value.
        - Create new_assignment by adding var = value.
        - **Check Validity:** If is_assignment_valid(new_assignment) is true:
            - **Forward Check:** Perform new_domains = forward_check(domains, var, value, new_assignment).
            - **Check for Empty Domains (after forward check):** Ensure no domain of an unassigned variable has become effectively empty (i.e., all its remaining values correspond to faction pairs already used in new_assignment). If any domain is empty, this path is a dead end.
            - If domains are still valid, recursively call backtrack(new_assignment, new_domains, depth + 1).
            - If the recursive call finds a solution, propagate it upwards.
    4. If no value for var leads to a solution, return None (backtrack).

6. Execution and Output

- **Pre-processing (AC-3):**
    - A copy of the initial domains is made.
    - AC-3 is run on this domains_copy to prune domains before starting the search.

    - The result and new domain sizes are printed.
- **Solving (Backtracking):**
    - The backtrack function is called with an empty initial assignment and the (potentially pruned by AC-3) domains_copy.
    - The time taken to find a solution is recorded.
- plot_schedule(solution_dict, ordered_day_time_slots, rooms_list):
  This function takes a valid solution (a dictionary) and generates a 2D plot visualizing the schedule.
    - It maps rooms to x-coordinates and combined Day-Time slots to y-coordinates.
    - Each cell in the grid corresponding to an assigned meeting (f1, f2, room) for a slot_key (e.g., "Day1_Morning") will display the names of the two factions meeting.
    - It uses matplotlib.pyplot to create the plot with appropriate labels, titles, gridlines, and styling. The Y-axis is inverted so "Day1_Morning" appears at the top.
- Displaying Solution:
  If a solution is found:
    - It prints the time taken.
    - It lists the faction pairs that will meet.
    - It displays the schedule day by day, and time slot by time slot.
    - It analyzes and prints faction participation (how many meetings each faction has).
    - It calls plot_schedule to visualize the schedule.
      If no solution is found, it prints "No solution found!".

Report: Strategic Diplomacy AI - Part A Adaptive Diplomatic Scheduling

1. CSP Formulation

The diplomatic summit scheduling problem is formulated as a Constraint Satisfaction Problem (CSP) with the following components:

- Variables (V):
  The variables are the 9 distinct time slots available for meetings. Each time slot is a unique combination of a day and a time of day.
    - Example: Day1_Morning, Day1_Afternoon, ..., Day3_Evening.
    - Let V=v1,v2,...,v9, where each vi represents a specific day-time slot.
- Domains (D):
  The domain for each variable vi is the set of possible meetings that can be scheduled in that time slot. A meeting is defined by a pair of unique factions and the room where they meet: (faction1, faction2, room).

- **Factions:** F=Alpha, Beta, Gamma, Delta, Epsilon, Zeta
- **Rooms:** R=RoomA, RoomB, RoomC
- **Neutral Rooms:** RN=RoomB⊂R
- **Faction Pairs:** P=(fa,fb)|fa,fb∈F,fa□=fb. The order does not matter, so (fa,fb) is the same as (fb,fa). Total of 15 unique pairs.
- The domain Di for a variable vi is initially constructed as: Di=(p,r)|p∈P,r∈R subject to the rival constraint:
  - If pair p=(fa,fb) is a rival pair (e.g., (Alpha, Delta)), then r must be in RN.
  - Otherwise (if p is not a rival pair), r can be any room in R.

- Constraints (C):
  The constraints define the conditions that a valid schedule (a complete assignment of variables to domain values) must satisfy:
  1. **C1: Unique Faction Pair Meetings:** No faction pair meets more than once across the entire schedule.
     - If vi is assigned (fa,fb,r1) and vj is assigned (fc,fd,r2) for i□=j, then the pair fa,fb must not be equal to the pair fc,fd.
  2. **C2: Faction Room Non-Consecutivity (Code's Interpretation):** A faction cannot have two of its own meetings, which are chronologically adjacent *in that faction's personal schedule*, in the same room.
     - For any faction fk∈F: Let Mk=[(d1,t1,r1),(d2,t2,r2),...,(dm,tm,rm)] be the chronologically sorted list of meetings faction fk participates in. Then for any j∈[1,m−1], rj□=rj+1.
  3. **C3: Faction Same-Day Break:** Factions must have at least one time slot break between their sessions on a single day.
     - If faction fk has a meeting in time slot (d,tx) and another meeting in (d,ty) (same day d), then the time difference |index(tx)−index(ty)| must be greater than 1. (e.g., Morning and Evening is fine; Morning and Afternoon is not).
  4. **C4: Rival Factions in Neutral Rooms:** Rival faction pairs can only meet in designated neutral rooms.
     - If vi is assigned (fa,fb,r), and (fa,fb) is a rival pair, then r must be in RN. (This is pre-filtered into the initial domains but is a fundamental constraint).
  5. **C5: One Meeting Per Time Slot:** Each time slot must be assigned exactly one meeting. (This is inherent in the CSP formulation where each variable is a time slot and seeks a single value).
  6. **C6: All Factions in a Meeting are Distinct:** Each meeting involves two *different* factions. (This is inherent in how faction pairs are generated).
  7. **C7: Two Negotiation Sessions Per Day (Objective, not strict constraint in code):** The problem objective states "2 negotiation sessions per day". The current code defines 3 time slots per day (Morning, Afternoon, Evening) and attempts to fill all 9 available slots across 3 days. This means some days will have 3 sessions. If strictly 2 sessions per day were required, one time slot per day

would need to be marked as unavailable or treated differently.

2. Solution Methodology

The CSP is solved using a backtracking search algorithm enhanced with several heuristics and consistency techniques:

- **Backtracking Search:** A depth-first search algorithm that incrementally builds a candidate solution. If an assignment violates a constraint, it backtracks.
- **Heuristics for Variable Selection:**
    - **Minimum Remaining Values (MRV):** Prioritizes assigning the variable (time slot) that has the fewest legal values remaining in its domain. The implementation counts values whose faction pairs have not yet been used in the current partial assignment.
- **Forward Checking:** After a variable is assigned a value, forward checking is used to prune the domains of unassigned variables by removing values that are inconsistent with the current assignment. This helps detect inconsistencies earlier.
- **Arc Consistency (AC-3):** Applied as a preprocessing step to the initial domains. AC-3 removes values from variables' domains that cannot be part of a consistent solution with a neighboring variable, thereby reducing the search space before backtracking begins.

3. Dynamic Handling: Room Destruction and Rescheduling (Conceptual)

The provided code (PartAA.py) successfully implements the CSP solver for finding an initial static schedule. However, it **does not currently include the dynamic handling module** for room destruction and subsequent rescheduling as specified in the "Dynamic Element" of the problem description.
Here's a conceptual approach to how this could be integrated:

- Detection:
  The system would need a mechanism to be notified or to detect that a room has become unavailable. This could be an external trigger or a periodic check after each completed time slot.
- Impact Assessment:
  Once a room (e.g., RoomX) is declared unavailable at a certain point in time (e.g., after Day 1, Morning session):
    1. Identify all meetings already scheduled in RoomX from the current time onwards in the solution.
    2. Past meetings in RoomX are unaffected.
- **Rescheduling Strategies:**
    1. **Minimal Perturbation (Constraint Repair / Local Search):**
        - **Identify Affected Assignments:** For each future meeting m = (f1, f2, RoomX) scheduled in the destroyed room at time slot ts:
            - This assignment (ts -> m) becomes invalid.

        - **Attempt 1: Change Room Only:** Try to find a new available and valid room RoomY for the pair (f1, f2) in the *same time slot* ts. This new assignment (ts -> (f1, f2, RoomY)) must satisfy all constraints, especially C2 (Faction Room Non-Consecutivity) for f1 and f2 with their other meetings, and C4 (Rival in Neutral) if applicable.
        - **Attempt 2: Move Meeting (Harder):** If changing the room in the same slot fails, the meeting (f1, f2) needs to be moved to an entirely different *available and unassigned* future time slot ts_new with a valid room RoomZ. This is more disruptive as it frees up ts and occupies ts_new.
    - **Iterative Repair:** If a change causes new violations, those must also be repaired. This can become complex. Techniques like min-conflicts could be used: for an invalidated assignment, try to change its room or time slot to minimize the number of new constraint violations. Iterate until a consistent state is reached or a threshold is met.
2. **Rollback and Re-Search (More Robust if Repair is Complex):**
    - **Invalidate and Unassign:** All future meetings scheduled in the destroyed room are invalidated. The time slots they occupied become unassigned. Potentially, all meetings from the point of failure onwards might need to be unassigned if repair proves too difficult.
    - **Update Constraints/Domains:** The destroyed room is permanently removed from the list of available rooms for all future domain generations or constraint checks.
    - **Selective Re-Search:** Re-run the backtracking solver for only the unassigned time slots (from the point of failure onwards). The already completed and valid part of the schedule before the failure remains fixed. The solver will work with the reduced set of available rooms.
    - This approach is generally more likely to find a solution if one exists, compared to complex local repair, but might change the schedule more significantly.

- **Implementation Considerations for Dynamic Handling:**
    - The main solution dictionary would need to be modifiable.
    - The list of rooms (and neutral_rooms if the destroyed room was neutral) would need to be updated.
    - The domains for future unassigned variables would need to be regenerated or filtered based on the new room availability.
    - A function, say handle_room_destruction(destroyed_room, current_solution, current_day_time_index), would:
        1. Update the global room list.
        2. Identify affected future assignments in current_solution.
        3. Unassign these slots.

4. Re-initialize domains for unassigned slots considering the removed room.
5. Call backtrack on the current_solution (which now has unassigned slots) and updated domains.

The challenge in dynamic rescheduling is to efficiently find a new valid schedule while minimizing disruption. The "without recomputing the entire schedule" suggests favoring repair or a very targeted re-search from the point of failure.

4. Assumptions Made in the Code (vs. Prompt Q2)

1. **Time Slots per Day (3 vs. Objective of 2):**
   - **Prompt Objective:** "2 negotiation sessions per day".
   - **Code Implementation:** Defines 3 time slots per day (Morning, Afternoon, Evening) and attempts to schedule meetings in all 9 available slots over 3 days. This means some (or all) days will have 3 sessions, not strictly 2. The code does not treat any time slot as inherently "off" or unavailable to meet the "2 sessions per day" objective.
2. **Interpretation of "No faction uses the same room in two consecutive sessions":**
   - **Prompt Constraint:** "No faction uses the same room in two consecutive sessions".
   - **Code's Interpretation (Constraint C2 in is_assignment_valid):** For any given faction, if you list their meetings chronologically (based on their *personal* schedule), no two *adjacent* meetings in *that faction's list* can be in the same room.
     - *Example:* If Faction Alpha's meetings are (Day1, Morning, RoomA) and (Day1, Evening, RoomA), the code flags this as invalid. This is a reasonable interpretation. An alternative, stricter interpretation could mean that if a faction is in RoomA in Day1_Morning, they cannot be in RoomA in Day1_Afternoon (the next global time slot), regardless of whether they have a meeting in Day1_Afternoon or who it's with. The code's interpretation is common for such constraints.
3. **Dynamic Element (Room Destruction) - Not Implemented:**
   - The provided code **does not** implement the dynamic rescheduling module for room destruction. The current script focuses on finding an initial, static schedule. The "Dynamic Handling" section above is a conceptual discussion.
4. **Neutral Room Exclusivity for Rivals:**
   - Rival pairs *must* use a neutral room. They cannot use non-neutral rooms. This is correctly implemented by filtering domains.
   - Non-rival pairs can use *any* room, including neutral ones. This is standard.
5. **Unique Faction Pairs & Meetings:**
   - "Each session involves 2 unique factions": Inherent in forming pairs.
   - "No faction meets another more than once": Correctly implemented by tracking pairs_used.

6. **Heuristics Implementation:**
    - **MRV:** Implemented in select_unassigned_variable by considering pairs not yet assigned.
    - **Degree Heuristic:** Mentioned in the problem description requirements but **not explicitly implemented** as a tie-breaker or primary criterion in select_unassigned_variable. The current selection is solely based on MRV.
    - **Forward Checking:** Implemented.
    - **Arc Consistency (AC-3):** Implemented as a preprocessing step.
7. **Output Format:**
    - The code produces a final schedule in a dictionary format and prints it.
    - It generates a 2D plot using matplotlib. The prompt asks for "Day × TimeSlot × Room ⟶ (Faction1, Faction2)", which the dictionary structure solution[day_timeslot] = (f1, f2, room) and the plot represent.
    - "CSP trace and domain size evolution": Domain sizes after AC-3 are printed for a few variables. A more detailed trace (e.g., variable selection order, domain changes during backtracking) would require more extensive logging within the search functions.
8. **Solver Completeness:** The backtracking algorithm implemented is complete; it will find a solution if one exists given the constraints and domain definitions.
9. **Single Solution:** The solver stops after finding the first valid solution. It does not attempt to find all possible solutions or an optimal one (though optimality isn't defined here beyond satisfying constraints).

Code Documentation for `Q2PartB.ipynb` - Strategic Resource Negotiation

This Jupyter Notebook implements the adversarial negotiation engine for Part B of the Strategic Diplomacy AI project. Each negotiation session scheduled in Part A becomes a zero-sum game between two factions. The engine uses Minimax with Alpha-Beta pruning and custom evaluation functions to determine strategies.

1. Setup and Definitions (Part B)

- **Libraries:**
    - `random`, `math`, `copy`, `time`: Standard Python libraries.
    - `numpy` (as `np`): For numerical operations, though not heavily used in the core game logic.
    - `matplotlib.pyplot` (as `plt`): For plotting results and game trees.
    - `collections.defaultdict`: Used in the `Faction` class for storing history.
- **Game Constants:**

- `ACTIONS = ['Offer', 'Threaten', 'Bluff', 'Concede', 'Delay']`: Defines the set of possible actions a faction can take during a negotiation round.
- `RESOURCES = ['Energy', 'Water', 'Technology']`: Defines the resources being negotiated.
- `MAX_URGENCY = 10`: Used for normalization in evaluation functions, representing the maximum possible urgency a faction can have for a resource.

2. Core Classes (Part B)

- **`Faction` Class:**
  - **`__init__(self, name, urgencies)`:**
    - `name`: The name of the faction (e.g., 'Alpha').
    - `urgencies`: A dictionary mapping each resource (from `RESOURCES`) to an integer value (0-10) representing the faction's need for that resource.
    - `fatigue`: Initialized to 0. This attribute is intended to increase if the faction has back-to-back sessions (as determined by Part A's schedule) and can affect negotiation utility.
    - `history`: A `defaultdict(int)` to store counts of past positive or negative encounters with other factions. This is used by the `eval_history` function.
  - **`__repr__(self)`:** Returns a string representation of the faction object (e.g., "Faction(Alpha)").
- **`NegotiationState` Class:** Represents an immutable state (a node) in the negotiation game tree.
  - **`__init__(self, p1, p2, turn, depth, max_depth, history=None, fatigue=None, last_action=None)`:**
    - `p1`, `p2`: `Faction` objects representing the two negotiating factions.
    - `turn`: An integer (0 or 1) indicating whose turn it is to act (0 for `p1`, 1 for `p2`).
    - `depth`: Current depth of this state in the game tree.
    - `max_depth`: The maximum depth for the depth-limited search.
    - `history`: A list of tuples `(turn, action)` representing the sequence of actions taken to reach this state. It's deep-copied if provided.
    - `fatigue`: A dictionary mapping faction names to their fatigue levels. It's initialized from the `Faction`objects if not provided.
    - `last_action`: The action that led to this state.
  - **`is_terminal(self)`:** Returns `True` if the state is a terminal node in the game tree. This occurs if: * The `depth` reaches `max_depth`. * The `last_action` was

'Concede' (and there's some history, implying it's not the very first move).
    - **next_player(self):** Returns the `Faction` object whose turn it is to act in the current state.
    - **opponent(self):** Returns the `Faction` object that is the opponent of the `next_player`.
    - **successors(self):** Generates a list of successor `NegotiationState` objects, one for each legal action from the current state.
        - It iterates through `ACTIONS`.
        - A simple rule is implemented: a faction cannot 'Bluff' twice in a row.
        - For each valid action, a new state is created with:
            - `turn` flipped (1 - `self.turn`).
            - `depth` incremented.
            - The new action added to the `history`.
            - `fatigue` and `max_depth` are carried over.

3. Evaluation Functions (Part B)

These functions estimate the utility of a terminal or depth-limited game state for the maximizing player. The problem requires three distinct custom evaluation functions.

- **_base_score(faction, opp, history):** A helper function used by the evaluation functions. It calculates a generic linear score:
    - Sum of the `faction`'s urgencies for all `RESOURCES`.
    - Minus 0.5 times the sum of the `opp` (opponent's) urgencies for all `RESOURCES`.
    - The `history` parameter is present but not used in this specific base score calculation, suggesting it's a placeholder for more complex base scores that might consider it.
- **eval_basic(state, maximizing_for):**
    - **Evaluation Design:** This is a purely static, urgency-based evaluation.
    - It identifies the maximizing player (`p_max`) and the opponent (`p_opp`) based on `maximizing_for` (0 for `state.p1`, 1 for `state.p2`).
    - Returns the `_base_score` for `p_max` against `p_opp`.
    - **Impact of Factors:**
        - **Resource Urgency:** Directly considered.
        - **Fatigue, Room Bias, Previous Encounters:** Not considered by this function.
- **eval_fatigue(state, maximizing_for):**
    - **Evaluation Design:** Considers resource urgency and penalizes for fatigue.
    - Calculates the `_base_score`.
    - Subtracts a `fatigue_penalty`, which is `state.fatigue[p_max.name] *`

`1.5`.
    - **Impact of Factors:**
        - **Resource Urgency:** Directly considered via `_base_score`.
        - **Fatigue:** Directly considered; higher fatigue for the maximizing player reduces their evaluated utility. The multiplier `1.5` determines the penalty's strength.
        - **Room Bias, Previous Encounters:** Not considered.
- **eval_history(state, maximizing_for):**
    - **Evaluation Design:** Incorporates trust/rivalry based on the history of actions in the current negotiation session.
    - Calculates the `_base_score`.
    - Calculates a `trust` score by iterating through `state.history`:
        - Actions by `p_max`:
            - 'Offer', 'Concede': Increase trust (+2).
            - 'Threaten': Decrease trust (-3).
            - 'Bluff': Decrease trust significantly (-4).
        - Actions by `p_opp` (opponent):
            - Weighed less (halved impacts compared to `p_max`'s actions on `p_max`'s trust).
            - 'Offer', 'Concede': Increase trust (+1).
            - 'Threaten': Decrease trust (-1.5).
            - 'Bluff': Decrease trust (-2).
    - Returns `_base_score + trust`.
    - **Impact of Factors:**
        - **Resource Urgency:** Directly considered via `_base_score`.
        - **Previous Encounters (Session History):** Directly considered. Cooperative actions increase utility, while aggressive/deceptive ones decrease it.
        - **Fatigue, Room Bias:** Not considered by this function.
        - **Partial Information:** This evaluation function does not explicitly model partial information about opponent urgencies; it uses the true urgencies stored in the `Faction` objects.

4. Minimax Algorithm with Alpha-Beta Pruning (Part B)

- **minimax(state, depth, alpha, beta, maximizing_player, eval_fn, stats):** Implements the Minimax algorithm with Alpha-Beta pruning for a zero-sum game.
    - `state`: The current `NegotiationState`.
    - `depth`: The remaining depth for the search.
    - `alpha`: The alpha value for pruning (best score for maximizer found so far on

the path to root).

- `beta`: The beta value for pruning (best score for minimizer found so far on the path to root).
- `maximizing_player`: An integer (0 or 1) indicating if the current player to move from `state` is the one whose utility we are trying to maximize in this call (relative to the root of this minimax call).
- `eval_fn`: The evaluation function to use for terminal or leaf nodes.
- `stats`: A dictionary to collect statistics (e.g., `nodes` expanded).
- **Logic:**
  - Increments `stats['nodes']`.
  - **Base Case:** If `depth == 0` or `state.is_terminal()`, it returns `(None, eval_fn(state, maximizing_player_at_root))`. Note: The `eval_fn` is called with respect to whose perspective the utility is being calculated at the root of the search for this turn. The `minimax` function itself alternates between maximizing for player 0 and player 1 (opponent) at different levels of the tree.
  - **Maximizing Player's Turn (player 0 at this state, assuming `maximizing_player` in the call refers to player 0 as the one we want the best action for):**
    - Initializes `max_eval = -math.inf` and `best_action = None`.
    - Iterates through `state.successors()`:
      - Recursively calls `minimax` for the successor state, decrementing `depth`, and switching to the minimizing player (player 1).
      - Updates `max_eval` and `best_action` if a better evaluation is found.
      - Updates `alpha`.
      - If `beta <= alpha`, prunes the remaining branches.
    - Returns `(best_action, max_eval)`.
  - **Minimizing Player's Turn (player 1 at this state):**
    - Initializes `min_eval = math.inf` and `best_action = None`.
    - Iterates through `state.successors()`:
      - Recursively calls `minimax` for the successor state, decrementing `depth`, and switching to the maximizing player (player 0).
      - Updates `min_eval` and `best_action` if a lower evaluation (better for the minimizer) is found.
      - Updates `beta`.

        - If `beta <= alpha`, prunes the remaining branches.
    - Returns `(best_action, min_eval)`.

5. Simulation and Output (Part B)

- **simulate_session(f1, f2, eval_fn, search_depth=4):** Runs a single negotiation session (one turn of the game from `f1`'s perspective).
    - Creates an initial `NegotiationState` (`root`) with `f1` to play (`turn=0`), `depth=0`, and `max_depth=search_depth`.
    - Initializes `stats = {'nodes': 0}`.
    - Calls `minimax` from the root state for player 0 (f1) to find the best action and its utility.
    - Records the time taken.
    - Returns a dictionary containing `first_action` (best action for f1), `utility` (for f1), `nodes` expanded, and `time` elapsed.
- **Demo Execution:**
    - Sets a random seed for reproducibility.
    - Creates two sample factions, `fA` and `fB`, with defined resource urgencies.
    - Defines a list of the three evaluation functions (`eval_fns`) and corresponding labels.
    - Iterates through each evaluation function:
        - Calls `simulate_session` for `fA` vs `fB`.
        - Prints the results: best first action for Alpha, Alpha's utility, nodes expanded, and time taken.
    - This demonstrates how each evaluation function might lead to a different initial strategic choice and perceived utility.
- **draw_small_tree(state, depth=2, eval_fn=None, maximizing_faction=None):**
    - **Game Tree Visualization Design:** A utility function to draw a small game tree (up to depth 2) using `matplotlib`. It's designed for illustrative purposes and does not use graph libraries.
    - **Functionality:**
        - Takes an initial `state`, `depth` (max 2), an optional `eval_fn`, and `maximizing_faction` (0 or 1, to show evaluated scores at terminal nodes from that faction's perspective).
        - Recursively traverses the game tree from the given `state` up to the specified `depth`.
        - Calculates positions for nodes and edges.
        - Terminal nodes are labeled 'T' and, if `eval_fn` is provided, with their evaluated utility.
        - Uses `matplotlib.pyplot` to draw nodes as circles with labels and

lines for edges.
    - **Output:** The notebook shows an example call to `draw_small_tree` for the `eval_basic` function, providing a visual representation of the initial part of the game tree.
    - **"Game tree screenshots":** This function provides the capability to generate such visuals. The screenshot would be the output plot from this function.
- **Benchmark of 3 Different Sessions:**
    - Defines three different pairs of factions for negotiation sessions.
    - For each session and for each of the three evaluation functions:
        - `simulate_session` is called.
        - Statistics (utility for the first player, nodes, time) are collected and printed.
    - The collected statistics are then put into a Pandas DataFrame.
    - A bar chart is plotted showing the 'Nodes expanded' for each evaluation function across the three different sessions. This helps compare the search effort.

6. Strategy Outcomes, Performance Metrics (Part B Analysis from Notebook)

- **Strategy Outcomes (First Action):** The demo output shows the first action chosen by Faction Alpha when playing against Faction Beta, for a search depth of 4, using each evaluation function:
    - `eval_basic` ⇒ first action: `Concede`, utility: +8.50
    - `eval_fatigue` ⇒ first action: `Concede`, utility: +8.50 (assuming fatigue was 0 for this initial demo)
    - `eval_history` ⇒ first action: `Concede`, utility: +9.50 (initial history is empty, so trust starts at 0, then actions build it)
    - *Note:* In this specific demo, all strategies lead to 'Concede' as the first optimal move for Alpha. This might change with different urgencies, fatigue levels, or deeper searches. The utilities differ, reflecting how each function values the state.
- **Performance Metrics:** The notebook collects and reports:
    - **Utility:** The evaluated utility for the first player (Alpha in the demo) from the root of the game tree. This indicates how good the chosen strategy is perceived to be by that evaluation function.
        - Basic: +8.50
        - Fatigue: +8.50
        - History: +9.50
    - **Nodes Expanded:** The number of game tree nodes visited during the Minimax

search. This is a measure of search effort.
    - Basic: 104 nodes
    - Fatigue: 104 nodes
    - History: 118 nodes (slightly more, likely due to history list copying/processing in `NegotiationState`and evaluation)
- **Time per Strategy:** The time taken for the Minimax search to complete for one session.
    - Basic: ~1.3 ms
    - Fatigue: ~1.0 ms
    - History: ~1.1 ms
- **Plots of Evaluation Performance:** The bar chart of "Search Effort per Evaluation Function" (nodes expanded) is a form of performance plot. The prompt also asks for "Plots of evaluation performance over time," which would typically involve running many self-play sessions and tracking how the evaluation functions' accuracy or the agent's win rate changes. This specific type of "over time" plot (related to learning/tuning) is not explicitly generated in the provided notebook cells, which focus on single-session simulations and comparisons. The "Learning-Based Evaluation Function (Required)" section of the prompt implies that such logs from self-play would be used for manual/statistical tuning, but the notebook primarily sets up the simulation framework.

7. Analysis: Impact of Fatigue, History, and Partial Info (Conceptual based on Design)

- **Impact of Fatigue (as designed in `eval_fatigue`):**
    - Fatigue is modeled as a numerical value for each faction.
    - The `eval_fatigue` function directly subtracts a penalty proportional to the maximizing player's fatigue (`fatigue_penalty = state.fatigue[p_max.name] * 1.5`).
    - **Effect:** Higher fatigue for a faction will lead to a lower evaluated utility for states where they are the maximizing player. This should make them favor actions that conclude negotiations faster or are less strenuous, or accept less favorable outcomes if their fatigue is high. The opponent's fatigue is not directly used by `p_max` in this evaluation but would be used by the opponent when it's their turn to evaluate.
    - The notebook's `Faction` class initializes fatigue to 0. To see its impact, one would need to set non-zero fatigue values before a session, presumably based on the schedule from Part A (e.g., if a faction has back-to-back sessions).
- **Impact of History (as designed in `eval_history` and `NegotiationState`):**

    - Negotiation history is tracked as a list of (player, action) tuples within a single session in `NegotiationState`.
    - The `eval_history` function modifies the base utility by adding a `trust` score.
    - Cooperative actions ('Offer', 'Concede') by either player increase this trust score (more so for the current player's own cooperative actions).
    - Aggressive ('Threaten') or deceptive ('Bluff') actions decrease the trust score (more so for the current player's own such actions).
    - **Effect:** A history of cooperative behavior will lead to higher evaluated utilities, potentially encouraging more 'Offer' or 'Concede' actions to maintain trust or achieve mutually (perceived) better outcomes. A history of aggressive/deceptive actions will lower utilities, possibly leading to more 'Threaten', 'Bluff', or 'Concede' (if trust is very low and prospects are poor).
    - The `Faction.history` attribute (a `defaultdict`) is intended for *inter-session* history (trust/rivalry from previous encounters with different factions), but `eval_history` as implemented in the notebook uses the *intra-session* `state.history`. To fully implement "previous encounters (trust or rivalry)" from the prompt, the `Faction.history` would need to be integrated into the evaluation, perhaps by adjusting the initial trust value in `eval_history` based on the specific opponent.
- **Impact of Partial Information (as per prompt, conceptual as not fully implemented in notebook's evaluation):**
    - **Prompt Requirement:** "Opponent does not know true resource priorities."
    - **Notebook Implementation:** The current evaluation functions (`eval_basic`, `eval_fatigue`, `eval_history`) all use the true urgencies of both `p_max` and `p_opp` (e.g., via `_base_score(p_max, p_opp, state.history)`which accesses `faction.urgencies` and `opp.urgencies`). This means they operate under **complete information** regarding resource priorities within the evaluation.
    - **To Implement Partial Information:**
        1. **Opponent Modeling:** Each faction would need to maintain a *belief* or *model* about the opponent's resource urgencies. This model could start uniform or based on stereotypes and be updated based on opponent actions during the negotiation (e.g., if an opponent strongly rejects an offer for 'Water', their urgency for 'Water' might be inferred as high).
        2. **Evaluation Function Modification:** The evaluation functions would need to use this *believed*opponent urgency rather than the true opponent urgency when calculating scores. For example, `_base_score` would take `faction.urgencies` and `faction.opponent_model[opp.name].urgencies`.

3. **Bluffing/Signaling:** Actions like 'Offer' or 'Bluff' would become more strategic. An 'Offer' might also signal one's own (potentially false) priorities. A 'Bluff' attempts to mislead the opponent about one's strength or intentions based on their assumed partial information.

- **Effect of Partial Info:** Negotiations become more complex. Players would try to infer opponent priorities while simultaneously trying to conceal their own or mislead. The effectiveness of 'Bluff' actions would heavily depend on the opponent's ability to see through them based on perceived inconsistencies or by calling the bluff. The game tree search would explore scenarios based on these beliefs.

- **Learning-Based Evaluation Function (Required by prompt, conceptual for notebook):**
  - **Prompt Requirement:** "Tune your evaluation logic using logs from self-play sessions (No ML libraries allowed; manual/statistical tuning only)."
  - **Notebook Status:** The notebook provides the framework for `simulate_session` which can be used for self-play. It collects metrics like utility and nodes expanded.
  - **Tuning Process:**
    1. **Logging:** Extend `simulate_session` or create a wrapper to log detailed game traces from many self-play sessions between factions using different evaluation functions (or variations of them). Logs should include states, actions taken, resulting utilities, and perhaps features of the game state (e.g., current fatigue levels, trust scores, resource disparities).
    2. **Parameter Adjustment:** The evaluation functions have parameters (e.g., the `0.5` multiplier for opponent urgency in `_base_score`, the `1.5` fatigue penalty, the +/- values for trust in `eval_history`).
    3. **Manual/Statistical Tuning:**
       - Analyze logs to see which actions/sequences lead to better outcomes (e.g., higher utility for a faction, or "winning" a negotiation if a clear win condition is defined beyond utility).
       - Adjust the weights/parameters in the evaluation functions based on these observations. For example, if 'Threaten' seems too powerful or too weak, adjust its impact on the `trust` score. If fatigue penalties are too harsh or too lenient, change the multiplier.
       - This is an iterative process: adjust parameters, run more self-play sessions, analyze, and repeat. The goal is to create evaluation functions that lead to more effective and plausible negotiation strategies.

- The current notebook sets up the basic evaluation functions; the tuning based on self-play logs would be the next step in development.

8. Assumptions Made in the Code (vs. Prompt Q2) - Part B

1. **Zero-Sum Game:** The Minimax implementation assumes a zero-sum game, where one faction's gain is the other's loss. The utility functions are designed from the perspective of the maximizing player.
2. **Action Set:** The `ACTIONS` list is fixed as per the prompt.
3. **Utility Factors - Implementation Status:**
   - **Resource Urgency:** Directly implemented in all evaluation functions.
   - **Fatigue:** Implemented in `eval_fatigue`. Assumes fatigue values are pre-set in `Faction` objects (e.g., from Part A's schedule).
   - **Room Bias: Not explicitly implemented** in any of the three evaluation functions. This would require the `NegotiationState` to know the room and evaluation functions to adjust utility based on whether the room is neutral or "owned" by one of the factions.
   - **Previous Encounters (Trust/Rivalry):** Partially implemented in `eval_history` through *intra-session* action history. Broader inter-session trust/rivalry (from `Faction.history`) is not directly used by the provided `eval_history` but the `Faction` class has a placeholder for it.
4. **Advanced Extensions - Implementation Status:**
   - **Multi-Round Memory (Faction History):** The `Faction.history` attribute exists, but its use for adjusting strategies based on *overall* negotiation history with an opponent (across multiple sessions or rounds if a session had multiple rounds) is not fully fleshed out in the evaluation functions shown. `eval_history`focuses on the current session's action sequence.
   - **Partial Information Gameplay: Not implemented** in the evaluation functions. They assume complete knowledge of opponent urgencies.
   - **Learning-Based Evaluation Function:** The framework for simulation exists. The actual *tuning* process based on self-play logs is a subsequent step not shown in the notebook's execution. The evaluation functions are currently hardcoded.
   - **Coalition Rounds:** Not implemented (marked as optional).
5. **Game Engine Implementation:**
   - **Minimax with Alpha-Beta Pruning:** Implemented.
   - **Depth-Limited Tree Search:** Implemented (`max_depth` in `NegotiationState` and `search_depth` in `simulate_session`).
   - **Three Distinct Custom Evaluation Functions:** Implemented (`eval_basic`, `eval_fatigue`, `eval_history`).

6. **Analysis of Admissibility and Consistency:** These are properties of heuristic functions in search algorithms like A*. For evaluation functions in Minimax, these terms are less directly applicable. The notebook focuses on empirical outcomes rather than formal proofs of such properties.
7. **Output Requirements:**
    - **Utility results per match:** Provided by `simulate_session` and demo.
    - **Nodes expanded and time per strategy:** Provided.
    - **Game tree visuals for at least 3 sessions:** The `draw_small_tree` function provides this capability for small depths. The notebook shows one example. More could be generated.
    - **Plots of evaluation performance over time:** The notebook plots nodes expanded (a measure of search cost). "Performance over time" in the sense of learning/tuning is not plotted.
8. **No ML Libraries:** The code adheres to this, using standard libraries and manual/statistical tuning approaches.
9. **Session Structure:** A "session" in Part B appears to be a single instance of `simulate_session`, which itself performs a depth-limited search to find the *first best action* for the starting player. A full game might involve multiple such turns/rounds, which is not explicitly modeled in the `simulate_session` loop but could be built on top of it. The `NegotiationState.history` tracks actions within one such search.
10. **Bluffing Rule:** A simple rule "cannot Bluff twice in a row" is implemented in `NegotiationState.successors()`. More complex bluffing dynamics (e.g., calling bluffs) are not explicitly part of the state transitions or evaluation beyond `eval_history` penalizing bluffs.